

תוכנית עבודה — Data_Project

פרויקט: Zybo Z7-20 על כרטיס DDR Data Logger
מטרת הפרויקט: בניית מערכת FPGA מדורגת הכוללת RTL ב-Verilog, סימולציות, Vitis C, DDR, AXI DMA, AXI-Stream, FIFO, CDC, ובהמשך Python/CSV/Graph.

מצב נוכחי: שלבים 0, 1 ו-2 הושלמו. השלב הבא הוא שלב 3 — Packet Builder FSM + Testbench.

Stage 0 — GitHub + Project Structure

סטטוס: הושלם

מטרה

להקים סביבת עבודה מסודרת ומקצועית לפרויקט, כך שכל הקוד, המסמכים, התמונות, הדוחות וקבצי הכלים יהיו מאורגנים במבנה שמתאים ל-GitHub ולתיק עבודות.

למה השלב חשוב

פרויקט FPGA מקצועי לא כולל רק קוד RTL. הוא צריך לכלול גם תיעוד, סימולציות, דוחות, תמונות, README ומבנה תיקיות ברור. זה מאפשר להציג את העבודה בצורה מסודרת למעסיקים ולהמשיך לפתח את הפרויקט בלי בלבול.

מה בוצע

- נוצרה תיקיית Data_Project.
- נוצר GitHub repository.
- נוצר README.md.
- נוצר מבנה תיקיות מסודר.
- נוצר מסמך תוכנית עבודה.
- נוספה סכמת בלוקים.
- בוצע commit ראשון.
- בוצע push ל-GitHub.

תוצר השלב

Repository מסודר הכולל python, vitis, tb, rtl, images, docs, README ו-reports.

Stage 1 — Data Protocol

סטטוס: הושלם

מטרה

להגדיר את פורמט חבילת הנתונים של הפרויקט לפני כתיבת ה-RTL.

למה השלב חשוב

לפני שבונים מערכת שמעבירה נתונים, צריך לדעת מה בדיוק עובר במערכת. פרוטוקול הנתונים מגדיר את המבנה של ה-Packet, כך שכל בלוק בפרויקט ידע מה לצפות לקבל ומה להעביר הלאה.

מה הוגדר

- DATA_WIDTH = 32
- Header
- Length
- Data sequence
- Checksum
- Expected Packet
- PASS / FAIL Criteria

תוצר השלב

קובץ docs/data_protocol.md שמגדיר את מבנה הנתונים של הפרויקט.

Stage 2 — Data Generator + Testbench

סטטוס: הושלם

מטרה

לבנות את בלוק ה-RTL הראשון בפרויקט: Data Generator, ולבדוק אותו באמצעות Testbench וסימולציה ב-Vivado.

למה הבלוק משמש

ה-Data Generator משמש כמקור נתונים פנימי ומבוקר. במקום לחבר בשלב מוקדם מקור חיצוני כמו SPI, UART, מצלמה או חיישן, הבלוק מייצר סדרת נתונים ידועה מראש בתוך ה-FPGA.

הסדרה שנבדקה:

1, 7, 8, 3, 6, 3

למה בחרנו בבלוק הזה

זה מאפשר לבדוק את צינור העברת הנתונים בצורה מבוקרת. כאשר אנחנו יודעים מראש מה נכנס למערכת, אפשר לבדוק בהמשך שהנתונים נשמרים באותו סדר דרך DMA, AXI-Stream, FIFO, Packet Builder ו-DDR.

מה בוצע

- הוגדרה מטרת הבלוק.
- הוגדרו כניסות ויציאות.
- נכתב rtl/data_generator.v.
- נכתב tb/tb_data_generator.v.
- בוצעה סימולציית Vivado behavioral simulation.
- נבדק רצף היציאה: 1, 7, 8, 3, 6, 3.
- נשמרה תמונת waveform.
- נוצר דוח סימולציה.
- בוצע commit.
- בוצע push.

תוצר השלב

בלוק Data Generator עובד ומאומת בסימולציה, כולל קוד RTL, קובץ Testbench, תמונת waveform ודוח סימולציה.

Stage 3 — Packet Builder FSM + Testbench

מטרה

לבנות FSM שמקבלת נתונים מה-Data Generator ומרכיבה מהם Packet מלא לפי הפרוטוקול שהוגדר בשלב 1.

למה הבלוק משמש

הבלוק מוסיף מבנה מסודר לנתונים. במקום להעביר רק מספרים גולמיים, הוא יוצר חבילה הכוללת Header, Length, Data ו-Checksum.

למה בחרנו בבלוק הזה

FSM הוא נושא מרכזי ב-FPGA וב-ASIC. בשלב הזה נתרגל תכנון מערכת מצבים אמיתית שמנהלת סדר פעולות ברור ומייצרת פלט לפי פרוטוקול.

תוצר צפוי

rtl/packet_builder_fsm.v, קובץ Testbench, וסימולציה שמוכיחה שה-Packet נבנה נכון.

Stage 4 — Sync FIFO + Testbench

מטרה

לבנות FIFO סינכרוני ולבדוק שמירה על סדר נתונים, כתיבה, קריאה, overflow, empty, full ו-underflow.

למה הבלוק משמש

FIFO משמש כחוצץ בין בלוקים. הוא מאפשר לבלוק אחד לכתוב נתונים ולבלוק אחר לקרוא אותם בקצב מתאים, בלי לאבד את הסדר.

למה בחרנו בבלוק הזה

זיכרונות ו-FIFO הם רכיבים בסיסיים בתעשיית FPGA/ASIC. זה שלב חשוב בהבנת זרימת נתונים בין בלוקים.

תוצר צפוי

rtl/sync_fifo.v, קובץ Testbench, וסימולציה שמוכיחה שה-FIFO שומר על סדר הנתונים.

Stage 5 — AXI-Stream Master + Testbench

מטרה

לבנות בלוק שמוציא נתונים בממשק AXI-Stream בסיסי.

למה הבלוק משמש

AXI-Stream הוא ממשק נפוץ להעברת נתונים רציפה בין בלוקים ב-FPGA, במיוחד כאשר עובדים עם AXI DMA.

למה בחרנו בבלוק הזה

זה מכין את הפרויקט לחיבור ל-AXI DMA ול-DDR, ומתרגל שימוש באותות כמו tvalid, tready, tdata ו-tlast.

.backpressure, וסימולציה הכוללת בדיקת rtl/axis_stream_master.v, קובץ Testbench.

Stage 6 — Full RTL Top Simulation

מטרה

לחבר את הבלוקים הראשונים יחד בסימולציה מלאה ברמת RTL.

למה השלב חשוב

לפני שעוברים ל-Block Design ולכרטיס, חשוב לבדוק שכל הבלוקים עובדים יחד בסביבה סימולטיבית.

תוצר צפוי

.data_project_top.v, קובץ Testbench מלא, וסימולציה שמראה מעבר נתונים מה-Data Generator ועד AXI-Stream Master.

Stage 7 — Vivado Block Design + AXI DMA + DDR

מטרה

לחבר את מערכת ה-RTL ל-AXI DMA, Zynq PS ול-DDR בתוך Vivado Block Design.

למה השלב חשוב

זה השלב שבו הפרויקט מתחיל להתחבר לחומרה של הכרטיס. הנתונים יוכלו לעבור דרך DMA ולהיכתב לזיכרון DDR.

תוצר צפוי

Block Design תקין, מחובר ומוכן להרצה על Zybo Z7-20.

Stage 8 — Vitis C Verification

מטרה

לכתוב תוכנת C ב-Vitis שמפעילה את ה-DMA, קוראת נתונים מה-DDR ובודקת שה-Packet תקין.

למה השלב חשוב

ב-Zynq יש שילוב בין PL ל-PS. שלב זה בודק את החיבור בין החומרה שכתבנו לבין התוכנה שרצה על המעבד.

תוצר צפוי

תוכנית C שמפעילה DMA, קוראת DDR ומבצעת בדיקת Header / Length / Data / Checksum.

Stage 9 — Async FIFO + CDC + Testbench

מטרה

להוסיף מעבר נתונים בין שני clock domains באמצעות Async FIFO.

למה השלב חשוב

CDC הוא נושא מרכזי בתעשייה. כאשר שני בלוקים עובדים עם שעונים שונים, אי אפשר להעביר נתונים ישירות בלי סנכרון מתאים.

תוצר צפוי

async_fifo.v, קובץ Testbench עם שני clocks, וסימולציה שמוכיחה מעבר נתונים תקין.

Stage 10 — Full Integration with CDC + DMA + DDR

מטרה

לשלב את ה-Async FIFO במערכת המלאה ולבדוק עבודה עם שני DMA, clock domains ו-DDR.

למה השלב חשוב

זה מחבר בין נושאי הליבה של הפרויקט: DMA, AXI, CDC, FIFO, FSM ו-DDR.

תוצר צפוי

מערכת מלאה עם שני domains וכתיבה תקינה ל-DDR.

Stage 11 — ILA + Vivado Reports

מטרה

להוסיף ILA לדיבוג חומרה אמיתי ולשמור דוחות Vivado.

למה השלב חשוב

בסימולציה רואים מה אמור לקרות. בעזרת ILA אפשר לבדוק מה באמת קורה על הכרטיס. בנוסף, דוחות timing ו-utilization חשובים להצגת איכות המימוש.

תוצר צפוי

ILA, timing report, utilization report, screenshots ותייעוד.

Stage 12 — Python CSV / Graph

מטרה

לייצא את הנתונים לקובץ CSV וליצור גרף בפייתון.

למה השלב חשוב

זה מוסיף שכבת הצגה וניתוח נוחה, ומראה יכולת לשלב בין חומרה, תוכנה וכלי ניתוח.

תוצר צפוי

CSV, קוד Python, גרף ותיעוד תוצאה.

Stage 13 — UART or SPI Extension

מטרה

להחליף את ה-Data Generator במקור נתונים אמיתי כמו UART או SPI.

למה השלב חשוב

זה הופך את המערכת ממקור נתונים פנימי למערכת שמסוגלת לקבל נתונים מבחוץ.

תוצר צפוי

בלוק תקשורת, בדיקה, וקבלת נתונים אמיתיים במערכת.

Stage 14 — Portfolio Polish

מטרה

ללטש את הפרויקט כך שיהיה מוכן להצגה למעסיקים.

למה השלב חשוב

תיק עבודות מקצועי צריך להציג לא רק קוד, אלא גם הסברים, תמונות, waveforms, דוחות, מסקנות ו-Future Work.

תוצר צפוי

GitHub מסודר, README משופר, תמונות, דוחות, הסברים ולקחים מהפרויקט.